Stokari · Complete 188-Task WBS Register

Target: Production Offline-First Retail Platform (SDD v0.4 & SRS v0.5) | **Structure:** 10 Isolated Single-Assignee Work Packages | **Total Issues:** 188 Discrete Deliverables

Pkg	Work Package & Subsystem	Primary Assignee Track	Tasks	Architectural Scope
PKG-01	**INFRA-DEVOPS**	Infrastructure & DevOps Engineer	**13**	Owns toolchains (uv/vite), CI pipelines, Postgres branching (Neon), observability (Sentry), rate limits, and automated disaster recovery tests.
PKG-02	**BACKEND-CORE-IDENTITY**	Backend Core & Identity Engineer	**13**	Owns common/kernel, common/persistence, Shop/User/Membership models, Google OIDC token auth, permissions, and shop lifecycle endpoints.
PKG-03	**CATALOG-PRODUCTS**	Catalog & Inventory Feature Engineer	**16**	Owns Product models, units catalogue, Latin+Ethiopic search folding, packaging conversions (crates/pieces), and catalog UI components.
PKG-04	**STOCK-LEDGER**	Ledger & Inventory Invariants Engineer	**24**	Owns append-only StockChange ledger, DB immutability triggers, deterministic row locking, Dexie stock tables, projection algorithm, and stock receiving/adjusting.
PKG-05	**POS-SALES**	POS & Sales Checkout Engineer	**16**	Owns Sale models, RecordSaleHandler, CancelSaleHandler, offline Dexie unit-of-work, cashier sell screen, cart stepper, decimal weight keypad, and payment selector.
PKG-06	**SYNC-PUSH**	Offline Sync Pipeline Engineer	**14**	Owns server push endpoint, per-shop row locks, ProcessedOperation idempotency ledger, Web Locks client manager, exponential backoff, and atomic Result Applier.
PKG-07	**SYNC-PULL**	Sync Pull & Client Hydration Engineer	**8**	Owns cursor-based delta pull endpoint, new-device bootstrap hydration, health/clock-skew probe, and header SyncPill component.
PKG-08	**CONFLICT-ARBITRATION**	Conflict Arbitration & Security Engineer	**16**	Owns ConflictCase models, StockBecameNegative event subscriber, 3 owner resolution actions, deactivated staff drain token flow, and review queue UI.
PKG-09	**DASHBOARD-REPORTS**	Management Experience & Reporting Engineer	**33**	Owns 3-bucket revenue breakdown, store valuation calculation, physical counts, local-only mode, session UI, staff leases, and emergency JSON export.
PKG-10	**DESIGN-SYSTEM-QA**	Design System & QA Lead	**35**	Owns design tokens, primitive UI components, two-device Playwright E2E tests, Hypothesis/fast-check property tests, trace_check gate, and visual regressions.

Note: Work packages are strictly decoupled by file boundaries to allow 1 person per package to work simultaneously without Git merge conflicts.

Package 1: Work Package 1: Repository Infrastructure, CI/CD & Production Hardening

Assignee: Infrastructure & DevOps Engineer | **GitHub Label:** `epic:infra-devops` | **Count:** 13 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#1	MUST	BACKEND	**Init stokari-api Django + uv, Ruff, mypy, import-linter, pytest, drf-spectacular**	Running CI pipeline, pyproject.toml locked with uv
#2	MUST	FRONTEND	**Init stokari-web Vite+React+TS + Vitest, Playwright, dependency-cruiser, ESLint**	Running CI pipeline, package.json with vp/vite
#3	MUST	INFRA	**GitHub Actions API CI: ruff, mypy, import-linter, file-size, pytest (Postgres), check --deploy, OpenAPI diff**	Green badge on main/dev branches
#4	MUST	INFRA	**GitHub Actions Web CI: lint, types, dependency-cruiser, tests, bundle-size, trace-check**	Green badge on main/dev branches
#5	MUST	INFRA	**Contract bundle workflow: export_contract.py, contract-release.yml, contract.lock, pnpm contract:sync**	Automated contract synchronization loop between backend and web
#6	MUST	INFRA	**Vercel (web) + Render (API) deployment pipelines with staging/prod**	Automated staging and production preview deployments
#7	MUST	INFRA	**Neon PostgreSQL dev/staging/prod branches + MIGRATION_DATABASE_URL**	Isolated database branches configured with zero-downtime migrations
#8	MUST	INFRA	**Sentry + UptimeRobot + nightly pg_dump cron + nightly reconciliation cron**	Observability and automated nightly logical backup scripts active
#143	MUST	BACKEND	**Django hardening: DEBUG=False, ALLOWED_HOSTS, no admin/auth/sessions/CSRF, browsable API disabled, SECURE_SSL_REDIRECT, HSTS, CSP**	Production settings configuration passing security checklist
#144	MUST	BACKEND	**Rate limits: push/pull 60 req/min/user, auth 10 req/min/IP (shared cache table across gunicorn workers)**	Distributed rate throttling protecting sync and auth endpoints
#145	MUST	BACKEND	**Body limits: 200KB push, 50 ops max; DATA_UPLOAD_MAX_MEMORY_SIZE hard cap**	Server upload payload limits protecting against memory exhaustion
#146	MUST	BACKEND	**Logging: JSON with requestId, shopId, opId only; no tokens/emails/payloads**	Structured log output scrubbed of sensitive PII and auth credentials
#147	MUST	INFRA	**Backup/restore test: staging restore from nightly dump → run reconciliation → verify**	Automated disaster recovery test passing ledger reconciliation

Package 2: Work Package 2: Backend Architecture, Identity, Tenancy & Permissions

Assignee: Backend Core & Identity Engineer | **GitHub Label:** `epic:backend-core-identity` | **Count:** 13 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#9	MUST	BACKEND	**common/kernel: Result, DomainError, typed IDs (ShopId, OpId, ProductId), Clock port, event bus**	Pure Python domain primitives with 100% unit test coverage
#10	MUST	BACKEND	**common/http: RFC 9457 handler, request-id middleware, throttles, StrictIntegerField, pagination**	Consistent error responses and request tracing across DRF views
#11	MUST	BACKEND	**common/persistence: TransactionManager (atomic + shop_lock), ShopScopedRepository, domain_event outbox**	Thread-safe and tenant-isolated data persistence base classes
#12	MUST	BACKEND	**apps/shops/domain/permissions.py → contract/fixtures/permissions.json**	Role-permission matrix exported to versioned JSON contract
#13	MUST	BACKEND	**Django models: Shop, Membership, User, OfflineLease, RefreshToken, Invite, Device (composite PKs)**	Initial migration with composite primary keys (shop_id, id)
#14	MUST	BACKEND	**Migrations 0001 (composite PKs) + 0002_protections (immutability triggers, composite FKs via RunSQL)**	Database level constraints and triggers active in PostgreSQL
#15	MUST	BACKEND	**AuthProvider + ActorAuthentication DRF class (verifies access JWT/drain token → Actor dataclass)**	Custom token auth resolving Actor without default Django user table
#16	MUST	BACKEND	**Endpoints: GET /auth/nonce, POST /auth/google, POST /auth/refresh, POST /auth/lease, POST /auth/logout**	Complete authenticated OIDC handshake and token life cycle API
#17	MUST	BACKEND	**Endpoints: POST /shops (idempotency key), GET /shops/me, POST /shops/me/close (confirmation flow)**	Shop registration, metadata inspection, and shop closure endpoints
#18	MUST	BACKEND	**Endpoints: POST /shops/me/invites, POST /invites/{token}/accept**	Invitation generator enforcing 5-attendant quota limit
#19	MUST	BACKEND	**Endpoints: GET /shops/me/members, POST /shops/me/members/{id}/deactivate**	Member list and instant deactivation taking effect immediately
#20	MUST	BACKEND	**Permission classes: ActiveMembership, ShopOpen, RolePermission (reads from fixture)**	DRF permission gates protecting tenant boundaries
#21	MUST	BACKEND	**Management commands: transfer_ownership, purge_closed_shop**	CLI operations tools for owner succession and compliance cleanup

Package 3: Work Package 3: Master Catalog, Products & Packaging Domain (Vertical Slice)

Assignee: Catalog & Inventory Feature Engineer | **GitHub Label:** `epic:catalog-products` | **Count:** 16 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#27	MUST	BACKEND	**Django Product model (composite PK, unit immutable, name/nameAlt, price/cost santim, lowStockMilli, status)**	PostgreSQL Product table with integer santim and milli-units
#28	MUST	BACKEND	**CatalogService + ProductReader ports; ProductRepository (ShopScoped)**	Domain service and scoped repository with automated query tests
#29	MUST	BACKEND	**Operations: CreateProduct (atomic with OPENING StockLog), UpdateProduct, SetProductStatus**	Idempotent product creation emitting initial opening ledger movement
#30	MUST	BACKEND	**Unit catalogue (apps/stock-log/domain/units.py) → contract/fixtures/units.json**	Canonical unit definitions (PIECE, KG, LITER, METER) exported to contract
#31	MUST	SHARED	**core/domain/quantity.py + core/domain/quantity.ts — parseQuantity, formatQuantity, validate_qty**	Bilingual unit parser and half-up rounding passing vector tests
#32	MUST	FRONTEND	**core/search/normalize.ts — Latin + Ethiopic homophone folding (ሀ/ሐ/ኀ, ሰ/ሠ)**	Phonetic Amharic text folding utility with sub-millisecond benchmark
#33	MUST	FRONTEND	**Web: Product-creation wizard ('How do you sell it?' → unit chips, preview sentence, 'Add another form')**	Step-by-step intuitive modal for adding products with packaging
#34	MUST	FRONTEND	**Web: QuantityInput component — keypad by unit kind, quick chips from quickQtyMilli, g/mL toggle for KG/L**	Touch numeric keypad component supporting fractional quantities
#35	MUST	FRONTEND	**Web: Product list/search (in-memory normalized index, Amharic fold, virtualization)**	Fast client search filter handling 2,000 SKUs with zero lag
#36	MUST	FRONTEND	**Web: Product detail (read-only unit badge, packaging editor [Stage 2])**	Product inspector showing prices, costs, barcodes, and stock level
#37	MUST	FRONTEND	**Web: Create/Edit Product modal (Zod validation mirroring backend)**	Modal form with margin calculator and strict boundary validation
#38	MUST	FRONTEND	**Web: Product card list (mobile) + DataTable (desktop) with density toggle**	Responsive dual-view UI with density toggles
#39	SHOULD	BACKEND	**Packaging rules [Should]: Product.packagings[], ReceiveLineInput.packaging, StockChange.packaging snapshot**	Packaging definition schema supporting crates, boxes, and cartons
#40	SHOULD	BACKEND	**Opening stock supports openingPackaging (count × factor)**	Product initialization accepting bulk container counts
#41	MUST	FRONTEND	**Product search toolbar: search input, category filter, status filter, 'Add Product' button**	Clean filter pill toolbar matching stitch prototype
#42	SHOULD	SHARED	**Category management (CRUD) [Should]**	Custom category taxonomy manager for shop owners

Package 4: Work Package 4: Stock Movement Ledger, Projections & Inventory Receiving

Assignee: Ledger & Inventory Invariants Engineer | **GitHub Label:** `epic:stock-ledger` | **Count:** 24 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#43	MUST	BACKEND	**Django StockLevel + StockChange models (composite PK, immutability trigger, underReview flag)**	Append-only stock ledger tables with database update prohibition trigger
#44	MUST	BACKEND	**StockLogService.append(entries, ctx) — locks StockLevel rows, increments version, writes StockChange, allocates serverSeq, returns crossed_below_zero**	Atomic append engine acquiring row locks in ascending product ID order
#45	MUST	BACKEND	**StockLogQueries: quantity, history, balance_at_version(baseVersion)**	Read-side queries optimized for fast ledger balance verification
#46	MUST	BACKEND	**Domain event: StockBecameNegative published in-transaction; ConflictService subscribes → opens/extends OFFLINE_OVERSELL case, sets underReview**	Event bus trigger detecting overselling race condition in-transaction
#47	MUST	BACKEND	**Nightly reconcile_stock_log management command (verifies qty = SUM(delta), version = COUNT)**	Automated verification job proving zero ledger balance corruption
#48	MUST	FRONTEND	**Web Dexie schema: stockChanges, stockLevels (projection with confirmedQty, pendingDelta, projectedQty)**	Local IndexedDB tables partitioned by shop and user identity
#49	MUST	FRONTEND	**Web StockLogRepository.appendPending(), recomputeFor(productIds) — projection algorithm (§5.10)**	Client projection engine computing balance with zero double-counting
#50	MUST	FRONTEND	**Web UnitOfWork (Dexie transaction wrapping sale + lines + stock log + outbox)**	Client atomic transaction wrapper ensuring all-or-nothing saves
#51	MUST	FRONTEND	**Web recovery: rebuildProjection(), integrity check (local confirmed = sum of confirmed entries)**	Disaster recovery routine rebuilding Dexie state from local ledger
#52	MUST	BACKEND	**Stock history endpoint: GET /products/{id}/stock-changes (role-filtered, paged)**	Paginated movement history API stripping cost fields for attendants
#53	MUST	FRONTEND	**Web: Stock screen card list (mobile) + DataTable (desktop) with Receive/Adjust/History actions**	Complete stock overview screen with quick-action buttons
#54	MUST	FRONTEND	**Web: StockLevelDisplay component — confirmed vs projected, low-stock threshold, unit badge**	Badge component visualizing stock health with warning indicators
#55	MUST	FRONTEND	**Web: Stock History drawer/sheet (chronological movements with actor, device, resulting balance)**	Slide-out audit drawer displaying immutable event trail and hashes
#56	MUST	FRONTEND	**Web: Receive Stock modal (packaging selector with live '3 crates = 36 pcs' preview)**	Shipment intake modal with live container unit calculations
#73	MUST	BACKEND	**ReceiveStockHandler (owner): multi-line, atomic, optional packaging per line, cost per unit or per container → updates product costSantim**	Atomic shipment receiver updating product cost and stock balances
#74	MUST	BACKEND	**AdjustStockHandler (owner): ADJUSTMENT entry with reason enum (DAMAGE, LOSS, CORRECTION, OTHER); online rejects**	Stock adjustment service enforcing mandatory

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
			negative, offline quarantines	reason codes
#75	MUST	FRONTEND	**Web: Receive Stock modal (segmented unit selector pc | Crate (12), live preview, optional cost per container)**	Shipment modal with live container-to-unit cost calculations
#76	MUST	FRONTEND	**Web: Adjust Stock modal (reason picker, quantity input by unit, preview)**	Adjustment modal requiring reason selection and delta confirmation
#77	MUST	SHARED	**Stock log entries carry packaging snapshot (label, count, baseQtyMilli) for history render**	Ledger snapshot schema preserving container details for audit trail
#78	MUST	FRONTEND	**Web: Adjust reason enum with mandatory note for OTHER**	Form validation rule requiring explanatory text when OTHER is chosen
#79	MUST	FRONTEND	**Web: Receive cost per container → converts to cost per base unit with half-up rounding**	Math utility dividing wholesale carton cost into unit cost
#80	MUST	FRONTEND	**Web: Stock screen quick actions bar (Receive/Adjust/Export)**	Sticky top action bar for inventory management workflows
#81	MUST	FRONTEND	**Web: Low stock threshold display on product cards**	Visual alert tag indicating when item drops below minimum threshold
#82	MUST	FRONTEND	**Web: Stock summary cards (Total SKUs, Low Stock Count)**	Summary metrics bar at the top of the inventory screen

Package 5: Work Package 5: POS Sales Register, Cart & Local Cashier Persistence

Assignee: POS & Sales Checkout Engineer | **GitHub Label:** `epic:pos-sales` | **Count:** 16 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#57	MUST	BACKEND	**Django Sale, SaleLine, SaleCancellation models (composite PKs, reversesId on cancellation)**	PostgreSQL sales schema with immutable line records
#58	MUST	BACKEND	**RecordSaleHandler: validates lines, recomputes totals, price snapshots, builds SALE entries, calls StockLogService.append, applies outcome matrix by mode**	Complete sales transaction engine with price snapshot preservation
#59	MUST	BACKEND	**CancelSaleHandler: inserts SaleCancellation (PK (shopId, saleId)), builds SALE_CANCELLATION entries with reversesId; quarantines on CANCELLATION_CONFLICT conditions**	Compensating cancellation handler with idempotency guard
#60	MUST	FRONTEND	**Web recordSale.ts (offline: one Dexie tx → sale, lines, stock log, projection, outbox; online: outbox only, await confirmation)**	INV-01 durable local persistence use case for cashier checkout
#61	MUST	FRONTEND	**Web cancelSale.ts (owner only, offline pending)**	Owner cancellation flow generating compensating outbox operation
#62	MUST	FRONTEND	**Web: Sell screen — search → tap product → quantity chips → payment label → Save (≤10s, ≤3 taps)**	High-speed cashier POS interface achieving sub-10-second checkout
#63	MUST	FRONTEND	**Web: Cart — line qty unit × price = lineTotal, rounded sum, 'Sell anyway' sheet when exceeding projected stock**	Cart management component with live subtotal and stock warnings
#64	MUST	FRONTEND	**Web: Payment Selector — segmented control (Cash/Telebirr/Other)**	3-button payment tender selector with high-contrast tactile feedback
#65	MUST	FRONTEND	**Web: Complete Sale button (54px sticky footer mobile, auto-width desktop)**	Ergonomic thumb-zone checkout CTA bar for mobile viewports
#66	MUST	FRONTEND	**Web: Quantity Keypad Sheet (40vh, numeric/decimal by unit, quick chips, live total preview)**	Slide-up touch keypad for entering arbitrary decimal weights (kg)
#67	MUST	FRONTEND	**Web: Sell Anyway Alert Sheet (warns confirmed vs projected, flags for review, destructive button)**	Explicit approval sheet allowing offline cashiers to oversell stock
#68	MUST	FRONTEND	**Web: Pending confirmation list (online) + 'Awaiting confirmation' badge**	Visual pending indicator showing sales awaiting server confirmation
#69	MUST	FRONTEND	**Web: Sale history (localDay EAT grouping, sync state badges)**	Transaction journal grouped by day with live sync status badges
#70	MUST	BACKEND	**Sale immutability: architecture test — UPDATE/DELETE on sale/stock log/cancellation fails**	Automated architectural test proving immutability cannot be bypassed
#71	MUST	FRONTEND	**Web: Product quick chips (horizontal scroll, max 6 frequent items)**	1-tap quick buttons for frequent staples (Sugar, Rice, Oil, Coffee)
#72	MUST	FRONTEND	**Web: Recent products persisted in meta.recentProducts**	Client persistence tracking top 6 cashier-selected items across reloads

Package 6: Work Package 6: Synchronization Engine — Push Pipeline & Idempotency

Assignee: Offline Sync Pipeline Engineer | **GitHub Label:** `epic:sync-push` | **Count:** 14 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#83	MUST	BACKEND	**PushService: per-op transaction, shop lock → product locks (ascending ID), re-check ProcessedOperation**	Deterministic lock ordering preventing deadlocks during sync push
#84	MUST	BACKEND	**Idempotency: ProcessedOperation table (opId, payloadHash, outcome, resultJson); same hash = stored result; different hash = IDEMPOTENCY_KEY_REUSE**	PostgreSQL idempotency ledger caching past operation results
#85	MUST	BACKEND	**OperationDispatcher registry (built in config/container.py): maps OperationType → (payload parser, handler); architecture test asserts completeness**	Decoupled operation routing registry with completeness check
#86	MUST	SHARED	**Payload hash: RFC 8785 canonical JSON of {type, payload} → SHA-256 → base64url; shared vectors in contract/fixtures/payload-hash-vectors.json**	Deterministic canonical hashing passing cross-platform test vectors
#87	MUST	BACKEND	**dependsOn resolution: dependent op gets REJECTED_DEPENDENCY or quarantined into same case**	Dependency resolver preserving operation ordering within batches
#88	MUST	BACKEND	**Lease validation at receipt time: membership ACTIVE, lease valid, device matches, op type allowed, clientCreatedAt in window ±5min**	Cryptographic lease validation enforcing temporal sync windows
#89	MUST	BACKEND	**Quarantine path: stores full payload, QUARANTINED outcome, AUTHORIZATION_REVIEW case, no stock log effect**	Quarantine storage isolating suspicious operations without ledger impact
#90	MUST	BACKEND	**Outcome matrix implementation (§6.8 table) for all operation types × situations**	Comprehensive outcome matrix covering all sync edge cases
#91	MUST	BACKEND	**ChangeLog allocation under shop lock: shop.syncSeq += 1 per touched entity; written to ChangeLog + StockChange.serverSeq + Product.updatedSeq**	Monotonic gap-free change sequence numbering per shop
#92	MUST	BACKEND	**Push response: per-op OperationResult with entries, stockAfter, conflictCaseId, reasonCode, replayed, noop**	Structured push response contract detailing exact per-op outcomes
#93	MUST	FRONTEND	**Web push stage: batch ≤50 ops, ≤200KB, ascending deviceSeq; retry same opId on transport failure; poison guard (8 failures → ATTENTION_REQUIRED)**	Client outbox dispatcher with poison pill quarantine protection
#94	MUST	FRONTEND	**Web backoff: exponential (base 2s, factor 2, cap 5min, full jitter)**	Network retry algorithm avoiding thundering herd server overload
#95	MUST	FRONTEND	**Web: Sync Engine single-flight Web Lock (stokari-sync-{shopId}) for multi-tab safety**	Web Locks API coordinator ensuring only one tab syncs at a time
#96	MUST	FRONTEND	**Web: Result Applier — updates Dexie rows in one atomic transaction (§5.10)**	Client sync consumer updating local records with confirmed server IDs

Package 7: Work Package 7: Synchronization Engine — Pull Pipeline, Bootstrap & Storage

Assignee: Sync Pull & Client Hydration Engineer | **GitHub Label:** `epic:sync-pull` | **Count:** 8 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#97	MUST	BACKEND	**PullService: GET /v1/sync/pull?cursor=N&limit=200 → next limit ChangeLog rows → load current entities (role-filtered) → return nextCursor, hasMore, headSeq, serverTime**	Cursor-based delta synchronization endpoint with role data-filtering
#98	MUST	BACKEND	**Pull response: products, stockChanges, stockLevels, sales, saleCancellations, actors, conflicts (owner only)**	Complete sync pull DTO contract with actor metadata
#99	MUST	FRONTEND	**Web pull stage: ONE Dexie tx — upsert by id, apply stockLevels only if version > confirmedVersion, recompute pendingDelta, update pullCursor**	Client pull processor updating local cache without overwriting pending ops
#100	MUST	FRONTEND	**New-device bootstrap: cursor=0 → pull pages until hasMore=false → integrity check → ready**	Initial device hydration loading entire store catalog onto clean device
#101	MUST	FRONTEND	**CURSOR_RESET handling: restart from 0 without touching outbox**	Recovery protocol recovering from expired server cursors gracefully
#102	MUST	BACKEND	**Health probe: GET /health (2s timeout, no DB heavy work); used for online/offline decision, warm-up**	Lightweight health check returning server clock for skew detection
#103	MUST	FRONTEND	**Web: Sync trigger schedule (app open, online event, tab visible, debounced 500ms, timer 30s)**	Event-driven sync coordinator triggering on connectivity return
#104	MUST	FRONTEND	**Web: SyncPill in header with popover (last sync, outbox count, storage, persistent status, 'Sync Now')**	Interactive status badge displaying 4 sync states and connection info

Package 8: Work Package 8: Conflict Detection, Security Quarantine & Owner Arbitration (AT-06)

Assignee: Conflict Arbitration & Security Engineer | **GitHub Label:** `epic:conflict-arbitration` | **Count:** 16 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#105	MUST	BACKEND	**ConflictCase model: type, productId, status, version, stateSnapshot, items[], quarantined[], resolution, resolvedBy**	PostgreSQL conflict arbitration case schema with optimistic locking
#106	MUST	BACKEND	**ConflictService: opens/extends cases on StockBecameNegative, PRODUCT_INACTIVE, CANCELLATION_CONFLICT, INVALID_STOCK_OPERATION**	Conflict manager grouping concurrent oversell events into single cases
#107	MUST	BACKEND	**ResolveConflictHandler (owner): validates caseVersion, applies RECONCILE_STOCK / CANCEL_SALE / ACKNOWLEDGE via StockLogService, closes case, clears underReview**	Owner resolution service executing corrective ledger transactions
#108	MUST	BACKEND	**Stale resolution: caseVersion mismatch → NEEDS_REVIEW + STALE_RESOLUTION event, case stays OPEN**	Optimistic concurrency guard preventing conflicting owner resolutions
#109	MUST	BACKEND	**ReviewQuarantineHandler (owner): ACCEPT_QUARANTINED (re-runs original handler as original actor, returns inner[] per-op results) / REJECT_QUARANTINED**	Quarantine review endpoint executing late operations retrospectively
#110	MUST	FRONTEND	**Web: Review queue (owner) — card list (mobile) + split-pane (desktop) with case list + detail drawer**	Owner arbitration dashboard displaying active conflict queue
#111	MUST	FRONTEND	**Web: Conflict case card — type badge, product, current balance, timeline preview (last 3 events)**	Summary card displaying conflicting registers and negative balance
#112	MUST	FRONTEND	**Web: Conflict case detail drawer — discrepancy alert, concurrent timeline, 3 resolution actions**	Parallel timeline comparison view showing Hana vs Dawit concurrent sales
#113	MUST	FRONTEND	**Web: Resolution actions — Record Restock (qty + note), Cancel Sale (select + reason), Acknowledge Deficit**	3 functional owner resolution action dialogs executing atomic fixes
#114	MUST	FRONTEND	**Web: Quarantined operations tab — table with Accept/Reject per operation or bulk per device**	Table inspecting operations sent by deactivated cashier devices
#115	MUST	FRONTEND	**Web: Quarantine Accept → re-runs handler, shows per-op results in inner[]**	Action executing quarantined operations into legitimate sales
#116	MUST	FRONTEND	**Web: Quarantine Reject → archives with reason, never deletes**	Action rejecting quarantined records with permanent audit record
#117	MUST	SHARED	**Notifications: CASE_OPENED, CASE_EXTENDED, QUARANTINE_PENDING → owner in-app + GET /notifications**	In-app notification system alerting owner to overselling events
#118	MUST	FRONTEND	**Web: Notifications screen (list + mark read)**	Notification drawer listing recent store alerts and sync warnings
#119	MUST	FRONTEND	**Web: Conflict state machine: OPEN → RESOLVED only via explicit ResolveConflict**	Client state machine preventing automatic or accidental case dismissal

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#120	MUST	BACKEND	**Architecture test: every ConflictType has resolution paths; no auto-close**	Automated architecture test verifying zero unhandled conflict states

Package 9: Work Package 9: Business Reports, Revenue Dashboard, Staff & Local Modes

Assignee: Management Experience & Reporting Engineer | **GitHub Label:** `epic:dashboard-reports` | **Count:** 33 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#22	MUST	FRONTEND	**Web: session management (tokens in memory, refresh in IndexedDB, lease check, clock-rollback guard)**	Client session layer guarding against clock tampering
#23	MUST	FRONTEND	**Web: Google Sign-In flow, Auth context (useAuth), authenticated route guards**	Google OAuth popup/redirect with protected router routes
#24	MUST	FRONTEND	**Web: Staff management screen (invite attendants, list members, deactivate)**	Owner UI showing lease countdowns and instant deactivation
#25	MUST	FRONTEND	**Web: Multi-shop selection screen (if user has >1 membership)**	Store switcher allowing owners to switch between retail locations
#26	MUST	FRONTEND	**Web: Create shop flow (first-time owner) + Accept invite flow (attendant)**	First-run onboarding wizards for new owners and hired attendants
#121	MUST	FRONTEND	**Offline dashboard (Web): three buckets — Confirmed (SYNCED), Local Pending (WAITING_TO_SYNC/SYNCING), Needs Review (NEEDS_REVIEW + quarantined) — no double counting**	Local revenue aggregation engine grouping sales into 3 distinct buckets
#122	MUST	BACKEND	**Server dashboard endpoint: GET /reports/dashboard?date= → DashboardReport (confirmed sales, needs-review sales, by payment label, low stock, open cases, quarantined count)**	Server financial summary endpoint aggregating daily retail metrics
#123	MUST	BACKEND	**Inventory valuation (owner): GET /reports/inventory-value → Σ max(qty,0) × costSantim over products with cost; approximate flag if any negative/under-review/uncosted**	SQL valuation query computing total store inventory capital value
#124	MUST	FRONTEND	**Web: RevenueBucket component — 3 cards (Confirmed=Emerald, Pending=Amber, Review=Crimson) with amount + count**	Visual financial KPI cards with distinct status colors and counts
#125	MUST	FRONTEND	**Web: ValuationCard — large total, helper line 'N items have no cost recorded; M items approximate'**	Owner valuation widget displaying total capital with uncosted warnings
#126	MUST	FRONTEND	**Web: Low stock warning cards — product name, current/threshold with unit, Restock button**	Stock replenishment cards with 1-tap shortcut to receive inventory
#127	MUST	FRONTEND	**Web: Today screen — date selector (EAT), 3 buckets, valuation (owner), low stock list, quick actions**	Full overview screen showing daily revenue, outbox status, and alerts
#128	MUST	FRONTEND	**Web: Dashboard density toggle (compact/comfortable table rows)**	Table density switcher for customizing view on mobile vs desktop
#129	SHOULD	BACKEND	**CSV exports [Should]: GET /exports/sales.csv, /stock-changes.csv (owner only)**	Streaming CSV export endpoints for tax accounting and bookkeeping
#130	MUST	FRONTEND	**Pending-records export (device): JSON stokari-pending-export/v1 from outbox/sales/stockChanges — never marked confirmed, sensitive data warning**	Client backup file generator exporting unsynced outbox records to JSON

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#131	MUST	FRONTEND	**Web: Export Pending Records button in Settings + Staff screens**	Functional export button triggering actual browser file download
#132	MUST	SHARED	**Date handling: Africa/Addis_Ababa timezone, localDay on sales for grouping**	Timezone normalizer ensuring transactions align to local Ethiopian date
#133	SHOULD	BACKEND	**CountSession + CountLine models [Should]**	PostgreSQL schema for physical inventory count sessions
#134	SHOULD	BACKEND	**SubmitCountHandler, DecideCountHandler [Should]**	Commands to record observed shelf counts and decide variances
#135	SHOULD	BACKEND	**Server proposes delta = observedMilli − balanceAtVersion(baseVersion) (stock log query); shows movements after baseVersion [Should]**	Variance reconciliation calculation accounting for intermediate sales
#136	SHOULD	BACKEND	**Owner edits delta, approves → COUNT_CORRECTION entry [Should]**	Owner approval workflow generating audit ledger adjustment movement
#137	SHOULD	FRONTEND	**Web: Count screen — start session, enter observed per product, review proposed delta, decide [Should]**	Interactive inventory count counting sheet with live delta feedback
#138	SHOULD	FRONTEND	**Web: Count session history [Should]**	List of historical count sessions and owner reconciliation approvals
#139	SHOULD	FRONTEND	**meta.mode = 'LOCAL_ONLY': no tokens, no outbox transmission, no pull; persistent banner [Should]**	Standalone mode operating purely offline without server authentication
#140	SHOULD	DOCS	**Explicit migration procedure to shared mode (out of MVP scope) [Should]**	Documented migration guide for upgrading standalone phone to cloud
#141	SHOULD	FRONTEND	**Data export/import for local-only → shared migration [Should]**	JSON bundle exporter and importer for transitioning local shops
#142	SHOULD	FRONTEND	**Web: Local-only mode toggle in Settings with warning [Should]**	Settings UI toggle switching application into unclouded operation
#153	MUST	FRONTEND	**Web: navigator.storage.persist() requested at first run; quota warnings in Settings**	Automatic persistent storage request preventing browser cache eviction
#154	MUST	FRONTEND	**Web: Persistent storage status shown in Settings (enabled/disabled)**	Diagnostic storage meter displaying disk usage and persistence badge
#155	MUST	FRONTEND	**Web: Clock skew banner (>5 min) with server time offset**	Warning banner alerting cashiers if phone clock drifts beyond 5 minutes
#156	MUST	FRONTEND	**Web: Lease expiry UX — banner at 24h, read-only after expiry, export still works**	Graceful lease expiration transitions disabling sales but allowing export

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#157	MUST	FRONTEND	**Web: Error toasts with retry action (no bare 'something went wrong')**	Actionable toast notifications with clear resolution steps
#158	MUST	FRONTEND	**Web: Empty states with CTA button (no illustrations)**	Minimalist, utilitarian empty states guiding users to create data

Package 10: Work Package 10: Design System Primitives, Testing Gates & Contract Verification

Assignee: Design System & QA Lead | **GitHub Label:** `epic:design-system-qa` | **Count:** 35 issues

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#148	MUST	QA	**Performance: Lighthouse + timers on target phone (2000 products, 50k movements); p95 ≤300ms local actions, search <1s, launch <3s, dashboard <2s**	Performance audit report meeting all NFR benchmarks on low-end hardware
#149	MUST	FRONTEND	**Bundle size: ≤2MB compressed (code splitting, lazy routes)**	Optimized production Vite build within strict 2MB network budget
#150	MUST	QA	**Accessibility: WCAG 2.1 AA, 44px touch targets, 320px no horizontal scroll, status = text + icon**	Axe-core audit report verifying zero accessibility violations
#151	SHOULD	SHARED	**i18n: all strings in en.json from day one; Amharic am.json [Should]**	Complete English string dictionary and bilingual Amharic fallback
#152	MUST	QA	**Usability test: ≥5 outside users, ≥80% complete basic sale on first attempt**	User research report proving cashier checkout usability in under 10 seconds
#159	MUST	SHARED	**Shared fixtures: money-vectors.json, quantity-vectors.json, payload-hash-vectors.json, permissions.json, error-codes.json, sample-operations/*.json**	Committed canonical fixture bundle shared between backend and web
#160	MUST	BACKEND	**API unit tests: money, quantity, status machines, permission table, conflict policy, projection**	Backend domain unit test suite achieving >90% test coverage
#161	MUST	BACKEND	**API integration tests (pytest-django + factory_boy + Postgres): every handler, repository, endpoint**	Database integration tests covering all repositories and endpoints
#162	MUST	BACKEND	**API concurrency tests: parallel pushes, duplicate opId, pull during push (transaction=True + threads)**	Multi-threaded race condition tests proving shop lock effectiveness
#163	MUST	BACKEND	**API property tests (Hypothesis): random interleavings → qty = Σ deltas, no duplicate effect, cursor safety**	Hypothesis property tests asserting ledger invariants across 1,000 runs
#164	MUST	BACKEND	**API architecture tests: one handler per op, no direct Model.objects, composite keys present, no business logic in models**	Import-linter rules asserting clean architectural boundaries
#165	MUST	FRONTEND	**Web unit tests (Vitest + fake-indexeddb): use cases, outbox, result applier, recovery, projection**	Client unit test suite achieving >90% test coverage in Vitest
#166	MUST	FRONTEND	**Web property tests (fast-check): projection and outbox state**	Fast-check property tests verifying client projection mathematical invariants
#167	MUST	FRONTEND	**Web contract tests: run every sample operation through Zod schemas; money/quantity vectors match Python**	Contract tests proving frontend TypeScript matches Python vectors exactly
#168	MUST	QA	**E2E tests (Playwright): two browser contexts as two devices; offline sale → sync → reload → conflict flow**	Automated two-device conflict simulation verifying AT-06 end-to-end
#169	MUST	QA	**Device tests: real low-end Android (2GB RAM), 320px emulator — install, offline open, NFR-01/02/14**	Low-spec device verification report confirming PWA installation and speed
#170	MUST	INFRA	**Traceability: @pytest.mark.req('FR-14', 'AT-04') + web test titles [FR-14][AT-04]; trace_check.py fails CI if Must req**	CI script failing pull requests if any Must requirement lacks a

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
			untagged	test tag
#171	MUST	DOCS	**AT-01..AT-20 mapping to test scenarios (T-01..T-35)**	Complete test matrix document linking requirements to test code
#172	MUST	INFRA	**OpenAPI diff in CI: fails if openapi.json stale vs code**	Automated CI check catching out-of-date API schemas before merge
#173	MUST	INFRA	**Coverage gates: 70% overall, 90% on domain/, core/, sync/**	Enforced code coverage thresholds in GitHub Actions
#174	MUST	QA	**Visual regression tests: all 4 states × 3 breakpoints × 2 roles per component**	Automated visual snapshot suite catching accidental CSS regressions
#175	MUST	QA	**Accessibility audit: axe-core in CI, manual screen reader test**	Axe-core gate ensuring zero accessibility errors in pull requests
#176	MUST	INFRA	**Performance regression: bundle size, TTI, sync latency tracked per PR**	Automated bundle and latency tracking on every pull request
#177	MUST	FRONTEND	**Design token implementation: CSS variables for all colors, spacing, radii, shadows, typography**	Central CSS tokens adhering strictly to utilitarian cobalt palette
#178	MUST	FRONTEND	**Component library: Button, Input, Badge, Card, Modal, Sheet, Drawer, DataTable with all variants**	Accessible primitive component library with 44px minimum tap targets
#179	MUST	FRONTEND	**Specialized components: SyncPill, QuantityInput, CurrencyInput, UnitSelect, StockLevelDisplay, PaymentSelector**	Retail-specific composite UI controls for currency and units
#180	MUST	FRONTEND	**Composite components: RevenueBucket, ConflictCaseCard, LeaseHealthRow, ProductCard, LowStockCard**	Card components displaying complex domain metrics cleanly
#181	MUST	FRONTEND	**Mobile Sell screen: header, search, quick chips, cart, stepper, payment selector, complete sale button**	Mobile cashier POS layout verified on 375px and 390px screens
#182	MUST	FRONTEND	**Mobile Quantity Keypad Sheet (numeric/decimal by unit, quick chips, live total)**	Touch decimal weight keypad optimized for fast one-handed entry
#183	MUST	FRONTEND	**Mobile Sell Anyway Sheet (destructive variant, confirmed vs projected)**	High-contrast warning dialog approving offline overselling
#184	MUST	FRONTEND	**Desktop sidebar (240px → 48px icon rail), sticky header, split-pane drawers**	Collapsible desktop navigation sidebar adapting to wide displays
#185	MUST	FRONTEND	**DataTable: sticky header, sortable columns, filter toolbar, bulk actions, density toggle**	High-density data table component with horizontal scroll wrapper
#186	MUST	FRONTEND	**Amharic text rendering: no baseline shift, proper font fallback (Noto Sans Ethiopic)**	Font stack verification eliminating Amharic glyph baseline distortion
#187	MUST	FRONTEND	**Reduced motion: pulse animations disabled, slide transitions instant**	prefers-reduced-motion CSS media query compliance

ID	Priority	Layer	GitHub Issue Title	Deliverable & Acceptance Criteria
#188	MUST	FRONTEND	**Dark mode: NOT SUPPORTED — verify light-only implementation**	Confirmation audit ensuring high-contrast light-only daytime theme